

Terraform: State Locking, Lock File & Destroy Protection

Quick reference guide for managing Terraform state safety and restricting destructive operations in shared/production environments

1. State Locking

A safety mechanism that prevents two people (or two CI/CD jobs) from running **terraform apply / plan / destroy** at the same time against the same state file.

When someone starts an operation, Terraform acquires a lock. Anyone else trying to run Terraform in parallel gets blocked with an "Error acquiring the state lock" message until the first operation completes. This protects against state corruption caused by concurrent writes.

How it's implemented:

- S3 backend → locking handled via a DynamoDB table (S3 alone doesn't support native locking)
- Terraform Cloud / Enterprise → locking is built in automatically
- Local backend → uses a local lock file on disk, but only protects you locally, not across a team

Example S3 backend with locking enabled:

```
terraform {
  backend "s3" {
    bucket = "my-tf-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
  }
}
```

2. Lock File (.terraform.lock.hcl)

A file auto-generated by **terraform init** that records the exact provider versions and checksums (e.g., AWS provider 5.42.0) used in your project.

It exists because providers get updated frequently. Without pinning, different team members running **terraform init** on different days could silently pull different minor/patch versions of a provider — causing inconsistent behavior or unexpected plan diffs.

What it guarantees:

- Everyone on the team and CI/CD gets the exact same provider version and binary checksum on **terraform init**
- Prevents supply-chain surprises — checksum verification ensures the provider binary wasn't tampered with
- Committed to git so it's shared consistently across the team

terraform init respects the existing lock file. **terraform init -upgrade** deliberately updates it to newer provider versions.

Side-by-side comparison

	State Locking	Lock File (.terraform.lock.hcl)
--	---------------	---------------------------------

Purpose	Prevent concurrent apply runs from corrupting state	Pin exact provider versions and checksums
Protects	The state file (infra record)	Provider binaries used across the team
Configured in	Backend config (e.g. DynamoDB table for S3 backend)	Auto-generated by terraform init, committed to git
Related to destroy protection?	No	No
Triggers on	Every plan / apply / destroy	Every init

Key point: Neither state locking nor the lock file has anything to do with restricting terraform destroy. That's a separate concern, covered below.

3. Restricting terraform destroy in Prod (or Any Env)

1. prevent_destroy lifecycle rule

Add to critical resources in config. Terraform hard-errors on any plan/apply that would destroy them — even with -auto-approve.

```
resource "aws_instance" "prod_db" {
  # ...

  lifecycle {
    prevent_destroy = true
  }
}
```

Caveat: only protects resources where this block is explicitly added — won't stop a destroy from wiping out other resources in the same state.

2. IAM restrictions / Service Control Policies (SCPs)

Deny destructive actions (ec2:TerminateInstances, rds>DeleteDBInstance, etc.) at the IAM role or AWS Organization level for prod credentials. Even a valid terraform destroy call gets rejected directly by AWS — the strongest layer since it's enforced outside Terraform entirely.

3. CI/CD-only applies

No local apply/destroy against prod state. Only run through a pipeline (GitHub Actions, Atlantis, Terraform Cloud) with a required manual approval step before anything destructive runs.

4. Sentinel / OPA policy-as-code (Terraform Cloud/Enterprise)

Enforce rules such as "no destroy actions allowed on resources tagged env=prod" automatically, before any apply is permitted to proceed.

5. Separate state per environment

Isolate prod into its own state file / backend / workspace so a destroy aimed at dev can't accidentally touch prod resources.

Practical recommendation: Most production-grade setups combine prevent_destroy on truly critical resources, IAM deny policies for destructive actions, and a CI/CD approval gate on any plan showing a destroy action —

rather than relying on any single control alone.